DATA STRUCTURES : UNIT – 2 (ANS'S)

1a.Define algorithm. Explain the process to find the efficiency of an algorithm with suitable examples

Algorithm

An **algorithm** is a **finite set of well-defined step-by-step instructions used to solve a problem or perform a specific task**. An algorithm should be **clear, precise, and must terminate after a finite number of steps**.

Example Algorithm (Finding the largest of two numbers)

1. Start
2. Read two numbers A and B
3. If A > B, print A is largest
4. Otherwise print B is largest
5. Stop

Efficiency of an Algorithm

The **efficiency of an algorithm** refers to how well the algorithm performs in terms of **time and memory usage**. It is mainly measured using **Time Complexity** and **Space Complexity**.

---

1. Time Complexity

Time complexity measures the **amount of time an algorithm takes to execute** depending on the size of the input.

Example:

```
for(i = 0; i < n; i++) {
   printf("%d", i);
}
```

Here the loop runs **n times**, so the time complexity is **O(n)** (linear time).

Another example:

```
for(i = 0; i < n; i++) {
  for(j = 0; j < n; j++) {
    printf("*");
  }
}
```

Here there are **two nested loops**, each running n times.
So total operations = **$n \times n = n^2$**

Time complexity = **$O(n^2)$**.

---

2. Space Complexity

Space complexity measures the **amount of memory required by an algorithm during execution**.

Example:

```
int a[10];
```

Here memory is allocated for **10 integers**, so the space complexity depends on the **size of the array**.

Another example:

```
int a, b, sum;
```

This program uses only **three variables**, so it requires **constant memory**, represented as **O(1)**.

1b.Describe with suitable example about quick sort algorithm(procedure, algorithm, program)

Quick Sort – Definition
Quick Sort is a **sorting algorithm that uses the divide and conquer method**. It works by **selecting a pivot element and arranging the elements so that smaller elements come before the pivot and larger elements come after it**. The same process is repeated for the sub-arrays until the list becomes sorted.

Procedure

1. Choose a **pivot element** from the array.
2. Arrange elements so that **smaller elements are placed before the pivot** and **greater elements after the pivot**.
3. Apply the same steps to the **left and right sub-arrays**.
4. Repeat until the array is **completely sorted**.

Example
Array:
25 10 30 15 20
After applying quick sort:
10 15 20 25 30

Algorithm

1. Start
2. Choose pivot element
3. Partition the array
4. Recursively apply quick sort to left part
5. Recursively apply quick sort to right part
6. Stop

C Program

```
#include<stdio.h>

void quicksort(int a[], int low, int high) {
   int i = low, j = high, pivot = a[low], temp;

   if(low < high) {
      while(i < j) {
         while(a[i] <= pivot && i < high)
            i++;
         while(a[j] > pivot)
            j--;
         if(i < j) {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
         }
      }

      temp = a[low];
      a[low] = a[j];
      a[j] = temp;

      quicksort(a, low, j-1);
      quicksort(a, j+1, high);
   }
}

int main() {
   int a[5] = {25,10,30,15,20};
   int i;

   quicksort(a,0,4);

   printf("Sorted array:\n");
   for(i=0;i<5;i++)
```

```
        printf("%d ",a[i]);

    return 0;
}
```

Output

Sorted array:
10 15 20 25 30

2b.Describe with suitable example about merge sort algorithm(procedure, algorithm, program)

Merge Sort

Merge Sort is a **divide and conquer sorting algorithm**. It works by **dividing the array into smaller sub-arrays**, sorting them, and then **merging them back together in sorted order**.

Procedure

1. Divide the list into **two equal halves**.
2. Recursively divide the sublists until **each sublist contains one element**.
3. Merge the sublists in a **sorted manner**.
4. Continue merging until the **entire list becomes sorted**.

Example

Array:
38 27 43 3 9 82 10
Divide:
38 27 43 3 | 9 82 10
38 27 | 43 3
38 | 27
After merging:
27 38
3 43
3 27 38 43
9 10 82
3 9 10 27 38 43 82
Sorted Array:
3 9 10 27 38 43 82

Algorithm

1. Start
2. If the list has **one element**, return it
3. Divide the list into **two halves**
4. Recursively sort the **left half**
5. Recursively sort the **right half**
6. Merge the two sorted halves
7. Stop

C Program

```
#include<stdio.h>

void merge(int a[], int l, int m, int r)
{
   int i=l, j=m+1, k=l, temp[50];

   while(i<=m && j<=r)
   {
      if(a[i] <= a[j])
         temp[k++] = a[i++];
      else
         temp[k++] = a[j++];
   }

   while(i<=m)
      temp[k++] = a[i++];

   while(j<=r)
      temp[k++] = a[j++];

   for(i=l;i<=r;i++)
      a[i] = temp[i];
}

void mergesort(int a[], int l, int r)
{
   int m;
   if(l < r)
   {
      m = (l+r)/2;
      mergesort(a,l,m);
      mergesort(a,m+1,r);
      merge(a,l,m,r);
   }
}

int main()
{
   int a[6] = {12,11,13,5,6,7};
   int i;

   mergesort(a,0,5);

   printf("Sorted array:\n");
   for(i=0;i<6;i++)
      printf("%d ",a[i]);

   return 0;
}
```

Output
Sorted array:
5 6 7 11 12 13

3a.Define ADT. Explain different type ADTs with their operations.

Abstract Data Type (ADT)
An **Abstract Data Type (ADT)** is a **logical model of a data structure** that defines **what operations can be performed on the data but not how these operations are implemented**. In other words, ADT describes **the behavior of a data structure** rather than its implementation.

Types of ADTs
ADTs are mainly classified into **two types**:

---

1. Linear ADT
In **Linear ADTs**, elements are arranged **sequentially (one after another)**. Each element has a **unique predecessor and successor** except the first and last elements.
Examples:
- **List**
- **Stack**
- **Queue**

Operations:
- **Insert** – add an element
- **Delete** – remove an element
- **Search** – find an element
- **Traverse** – access elements sequentially

Example:
Stack operations → **push(), pop(), peek()**

---

2. Non-Linear ADT
In **Non-Linear ADTs**, elements are **not arranged sequentially**. Instead, they are connected in a **hierarchical or network structure**.
Examples:
- **Tree**
- **Graph**

Operations:
- **Insert node**
- **Delete node**
- **Search node**
- **Traversal** (preorder, inorder, postorder in trees)

Example:
Tree operations → **insert(), delete(), traverse()**

3b.Describe with suitable example about Radix sort algorithm(procedure, algorithm, program)

Radix Sort

Radix Sort is a **non-comparative sorting algorithm** used to sort numbers by **processing individual digits**. It sorts numbers **digit by digit**, starting from the **least significant digit (LSD)** to the **most significant digit (MSD)**.

Procedure

1. Find the **maximum number** in the list to know the number of digits.
2. Sort the numbers based on the **least significant digit (units place)**.
3. Then sort based on the **tens place digit**.
4. Continue sorting for **hundreds, thousands, etc.**
5. Repeat the process until all digits are processed and the array becomes **sorted**.

Example

Array:
170 45 75 90 802 24 2 66
After sorting by **units place**:
170 90 802 2 24 45 75 66
After sorting by **tens place**:
802 2 24 45 66 170 75 90
After sorting by **hundreds place**:
2 24 45 66 75 90 170 802
Sorted Array:
2 24 45 66 75 90 170 802

Algorithm

1. Start
2. Find the **maximum element** in the array
3. For each digit position (units, tens, hundreds)
4. Apply **counting sort** based on that digit
5. Repeat until the **highest digit is processed**
6. Stop

C Program

```
#include<stdio.h>

int getMax(int a[], int n) {
   int max = a[0];
   for(int i=1;i<n;i++)
      if(a[i] > max)
         max = a[i];
   return max;
}

void countSort(int a[], int n, int exp) {
   int output[20], count[10]={0};

   for(int i=0;i<n;i++)
```

```
        count[(a[i]/exp)%10]++;

    for(int i=1;i<10;i++)
        count[i] += count[i-1];

    for(int i=n-1;i>=0;i--) {
        output[count[(a[i]/exp)%10]-1] = a[i];
        count[(a[i]/exp)%10]--;
    }

    for(int i=0;i<n;i++)
        a[i] = output[i];
}

void radixSort(int a[], int n) {
    int max = getMax(a,n);

    for(int exp=1; max/exp>0; exp*=10)
        countSort(a,n,exp);
}

int main() {
    int a[] = {170,45,75,90,802,24,2,66};
    int n = 8;

    radixSort(a,n);

    printf("Sorted array:\n");
    for(int i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
}
```

Output

Sorted array:
2 24 45 66 75 90 170 802

4a.Describe with suitable example about merge sort algorithm(procedure, algorithm, program)

Merge Sort

Merge Sort is a **divide and conquer sorting algorithm**. It divides the array into **two halves**, sorts them separately, and then **merges the sorted halves** to produce a sorted array.

Procedure

1. Divide the array into **two equal halves**.
2. Continue dividing until **each sub-array has only one element**.

3. Merge the sub-arrays in **sorted order**.
4. Repeat merging until the **entire array is sorted**.

Example
Array:
12 11 13 5 6 7
Divide:
12 11 13 | 5 6 7
12 | 11 13
11 | 13
Merge:
11 13
11 12 13
5 6 7
5 6 7 11 12 13
Sorted Array:
5 6 7 11 12 13

Algorithm
1. Start
2. If the array contains **only one element**, return it
3. Find the **middle position**
4. Divide the array into **left and right parts**
5. Apply merge sort to both parts
6. Merge the two sorted parts
7. Stop

C Program

```
#include<stdio.h>

void merge(int a[], int l, int m, int r)
{
   int i=l, j=m+1, k=l, temp[50];

   while(i<=m && j<=r)
   {
      if(a[i] <= a[j])
         temp[k++] = a[i++];
      else
         temp[k++] = a[j++];
   }

   while(i<=m)
      temp[k++] = a[i++];

   while(j<=r)
      temp[k++] = a[j++];

   for(i=l;i<=r;i++)
      a[i] = temp[i];
```

```
}

void mergesort(int a[], int l, int r)
{
   int m;
   if(l < r)
   {
      m = (l+r)/2;
      mergesort(a,l,m);
      mergesort(a,m+1,r);
      merge(a,l,m,r);
   }
}

int main()
{
   int a[6] = {12,11,13,5,6,7};
   int i;

   mergesort(a,0,5);

   printf("Sorted array:\n");
   for(i=0;i<6;i++)
      printf("%d ",a[i]);

   return 0;
}
```

Output

Sorted array:
5 6 7 11 12 13

4b.Define algorithm. Explain the process to find the efficiency of an algorithm with suitable examples

Algorithm

An **algorithm** is a **finite sequence of well-defined step-by-step instructions used to solve a problem or perform a specific task**. An algorithm should be **clear, unambiguous, and must terminate after a finite number of steps**.

Example Algorithm (Addition of two numbers):

1. Start
2. Read two numbers A and B
3. Compute Sum = A + B
4. Display Sum
5. Stop

Efficiency of an Algorithm

The **efficiency of an algorithm** refers to how well an algorithm performs in terms of **execution time and memory usage**. It is mainly evaluated using **Time Complexity** and **Space Complexity**.

---

1. Time Complexity

Time complexity measures the **amount of time required by an algorithm to run as the input size increases**.

Example:

```
for(i = 0; i < n; i++)
{
   printf("%d", i);
}
```

This loop runs **n times**, so the time complexity is **O(n)** (linear time).

Another example:

```
for(i = 0; i < n; i++)
{
   for(j = 0; j < n; j++)
   {
      printf("*");
   }
}
```

Here the loop runs **n × n = n² times**, so the time complexity is **O(n²)**.

---

2. Space Complexity

Space complexity measures the **amount of memory used by an algorithm during execution**.

Example:

```
int a[10];
```

Here memory is allocated for **10 integers**, so the space complexity depends on the **size of the array**.

Another example:

```
int a, b, sum;
```

This program uses only **three variables**, so it requires **constant memory**, represented as **O(1)**.